

RookDB Assessment

[Aviral Gupta (2023111023)]

1. What happens when RookDB is started using cargo run?

'cargo run' is like './a.out' in Rust language. It triggers the main function in `src/main.rs`, which then calls the `run()` function in `frontend/menu.rs`. This function prints the welcome message, initialises the buffer manager and starts the main execution loop (of displaying the menu, waiting for user input and executing it).

2. Explain how a new table is created in RookDB. What steps are involved in creating a table? How many pages are created initially for that table?

The table creation begins with the user inputting the column headers and their respective data types. After which, the 'create_table' function in 'src/backend/catalog/catalog.rs' is called, which validates the input, updates the 'catalog' struct with the table details and also updates `catalog.json`. A dedicated file ('database/db_name/table_name.dat') is also created for persistent storage of data and headers.

A new table starts with two pages:

- Page 0 : The Table Header Page, containing metadata.
- Page 1 : The 'first' (and empty) Data Page.

3. How are rows (tuples) inserted into a table file in RookDB?

In the current implementation, one can only insert data by loading data through a CSV file. The 'load_csv_cmd' func in 'src/frontend/data_cmd.rs' calls 'buffer_manager.load_csv_to_buffer', which reads the CSV file, converts each row into binary format, places it into an in-memory page using a slotted page structure, and then writes the page to the table file on disk.

4. What information is stored in the page header, and why is it required?

Each page has an 8 byte header, comprising of two 4-byte unsigned integers / pointers:

- Lower Pointer : Essentially points to the beginning of the free space from the lower end of the page. When a new tuple is added, a new item pointer is added at this offset, and the lower pointer is incremented.

- Upper Pointer : Essentially points to the end of the free space from the upper end of the page. When a new tuple is added, its data is written at this offset (minus the size of the tuple), and the upper pointer is updated to the new start of the tuple data area.

These pointers are key to managing the memory landscape in the slotted page design (more info in Q7), enabling easy insertion and deletion without any expensive data movement.

5. Explain how free space is calculated within a page.

The free space within a slotted-page is calculated using the 'page_free_space' function in 'src/backend/page/mod.rs', which makes use of the upper (U) and lower (L) pointers and calculates the total free space of the page using the simple formula: $(\text{free_space} = U - L)$.

6. How does RookDB track the total number of pages in a table file?

The first 4 bytes of every table header file is reserved for a 32-bit unsigned integer that represents the total number of pages in that file.

7. What is a slotted-page layout? How is the number of rows (tuples) in a page determined?

The slotted-page layout is a commonly used technique to organise records (tuples) within a data page in a database system. A page is organised as follows:

1. Page Header: The first 8 bytes of the page, containing the lower and upper pointers for memory management.
2. Slot Array (Item Pointers): Immediately following the header, there is an array of item pointers. Each item pointer contains two pieces of information about a tuple: the offset within the page where the tuple's data begins and the length of the tuple's data. This array grows from the beginning of the page towards the end.
3. Free Space: The contiguous block of empty space between the end of the slot array and the beginning of the tuple data.
4. Tuple Data: The actual data for the tuples is stored at the end of the page. This data area grows backwards, from the end of the page towards the beginning.

This layout is advantageous because it allows for variable-length tuples without requiring costly data movement when tuples are deleted or updated.

The number of tuples in a page is determined by the number of item pointers in the slot array, which can be found using the formula: $\text{'num_of_tuples'} = (\text{lower_pointer} - \text{PAGE_HEADER_SIZE}) / \text{ITEM_ID_SIZE}$

8. What limitations exist in the current storage design of RookDB?

The current storage design of RookDB is very basic and has several significant limitations that would prevent it from being used in a real-world scenario. Some of these are as follows:

1. **No Indexing:** The database only supports heap files. This means that to find a specific row or set of rows, the system must perform a sequential scan of every page in the table. This is extremely inefficient for queries on large tables.
2. **No Transactions or ACID Guarantees:** The system lacks a transaction manager. Operations are not atomic, and there is no support for rollbacks. A crash during an operation (like loading a CSV) could leave the table file in a corrupt or inconsistent state.
3. **Insert-Only Operations:** The only way to get data into the database is by loading a CSV file. There is no support for 'DELETE' or 'UPDATE' operations on existing rows.
4. **Inflexible Data Types:** Apart from the very limited data types, the 'TEXT' data type is implemented as a fixed-size 10-byte array. Any text longer than 10 bytes is truncated, and any text shorter is padded with spaces. This is highly inefficient and impractical.
5. **No Support for Schema Evolution:** It is not possible to alter a table's schema (e.g., add or remove a column) after it has been created.

9. Are there any architectural or design issues in the existing design / code?

While a deeper understanding of the project would reveal the intricacies in the project and codebase, few flaws that I could notice from an overview glimpse are as follows:

1. **Tight Coupling:** The layers of the database are not well-separated. The `frontend` (user interface) directly interacts with low-level storage components like the 'BufferManager' and 'Catalog'.
2. **Inadequate Error Handling:** The code frequently uses 'unwrap()' and 'expect()', which will cause the program to panic on unexpected input or I/O errors.
3. **Hardcoded Values:** File paths (like 'database/global/catalog.json') and other constants are hardcoded throughout the application.

10. What improvements would you suggest to improve RookDB overall?

To improve RookDB, several areas could be addressed:

1. Indexing

2. Full DML support for INSERT, UPDATE, and DELETE operations should also be implemented.
3. More data types are supported such as BOOLEAN, FLOAT, DATETIME, DATE, VARCHAR, etc.
4. Robust Error Handling
5. Transaction and concurrency control system (using locking and logging) for ACID compliance and multi-user safety.